



第6章

深度学习计算系统

Why MLSys?



ResNet
Transformer

ML Research

44k lines of code

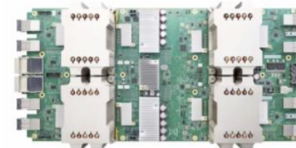
Six months

IMAGENET

Data



Compute



Why MLSys?



ResNet
Transformer

ML Research

100 lines of python

A few hours

System Abstractions

Systems (ML Frameworks)



IMAGENET

Data

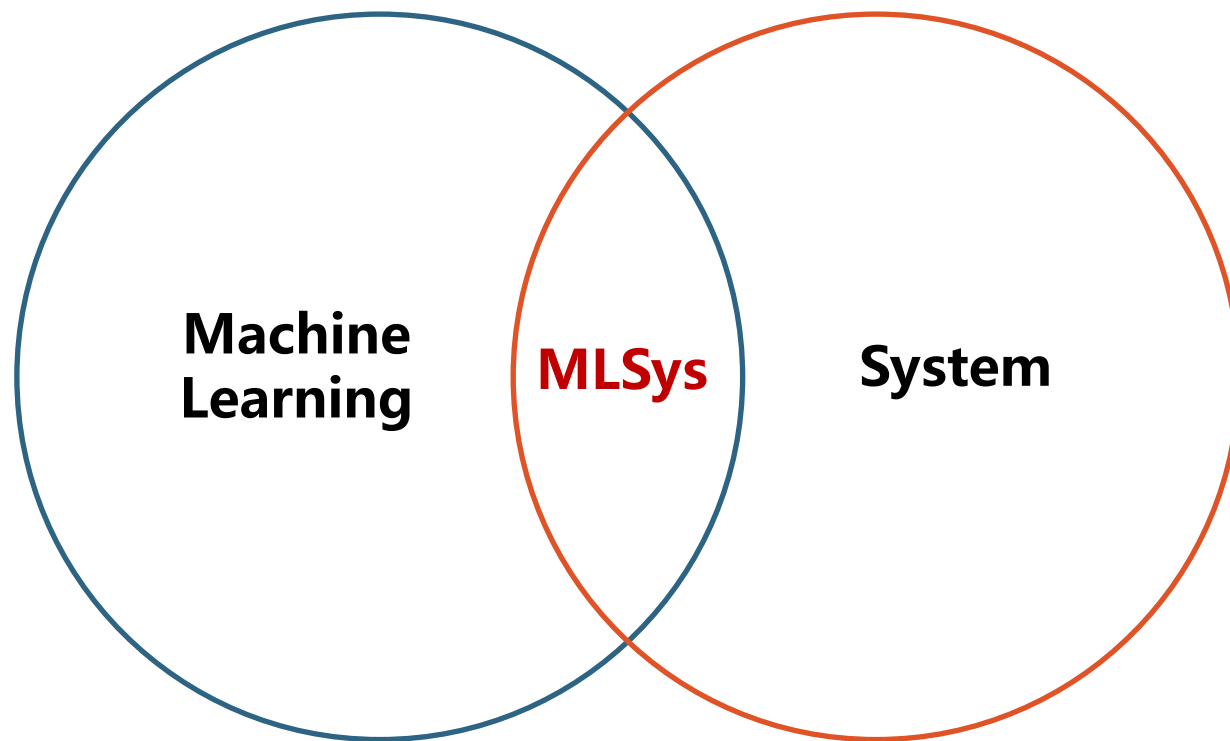


Compute



Why MLSys?

- **机器学习系统 (Machine Learning Systems, MLSys) 是一个新兴的交叉学科领域，核心在于研究如何设计、构建和优化能够支撑大规模机器学习应用的系统。**
 - 连接前沿机器学习理论与实际工程应用的桥梁。



机器学习系统的研究范畴



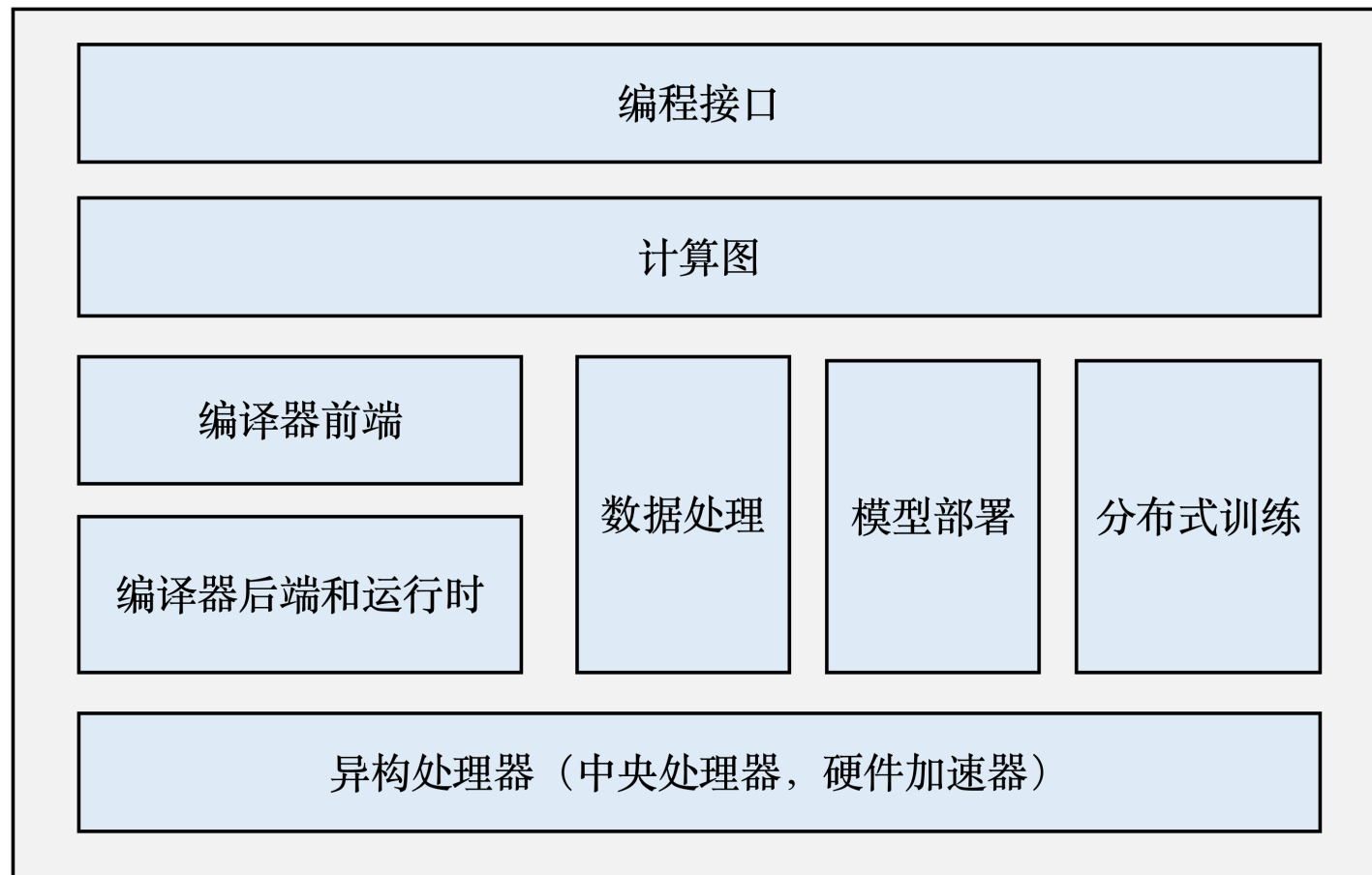
研究范畴

- **MLSys的范畴主要包括：**

- 神经网络编程
- 自动微分
- 数据管理和处理
- 模型训练和部署
- 硬件加速器
- 分布式执行

- **MLSys的组成：**

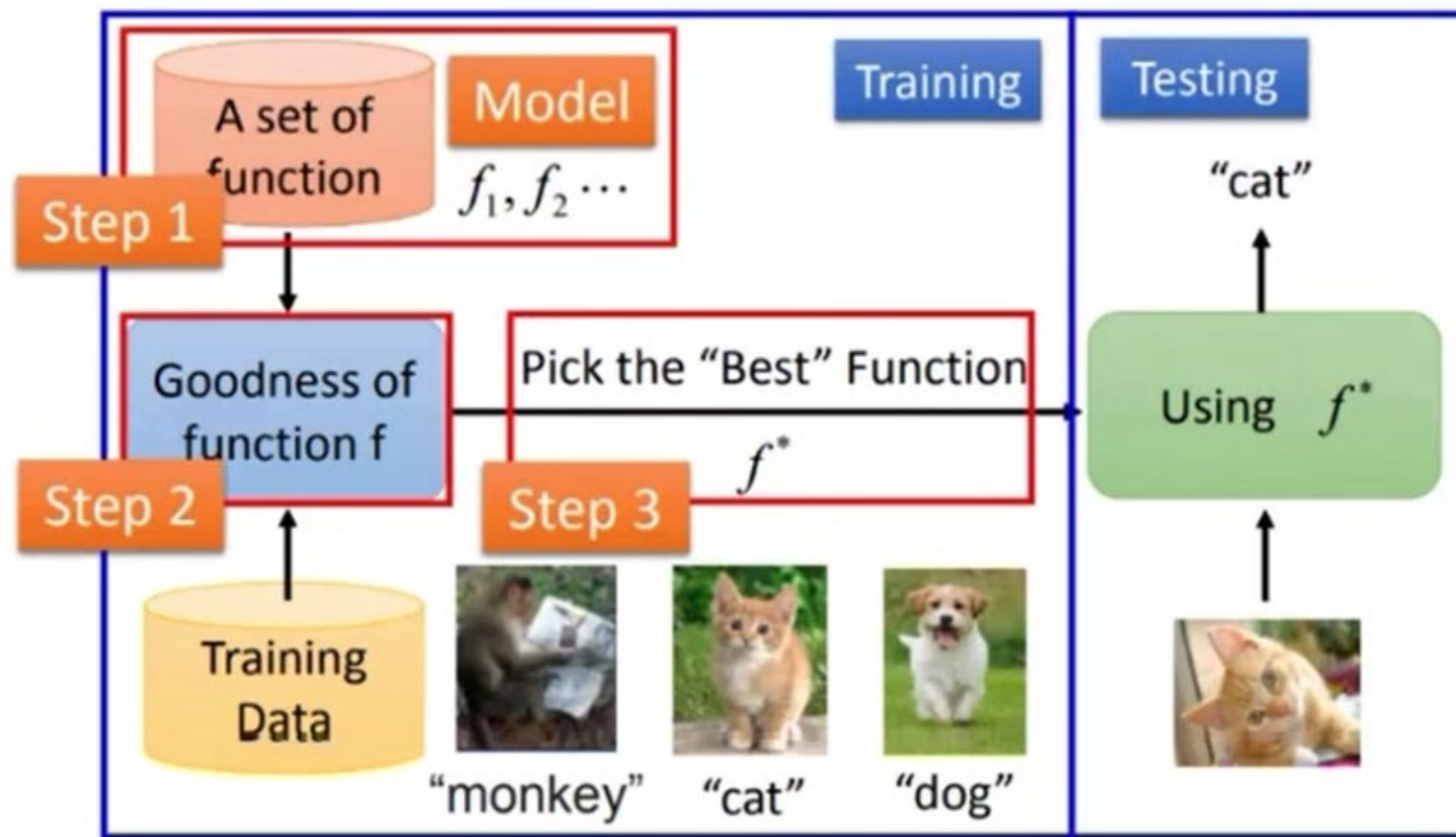
- 编程接口与计算图
- 编译器
- 异构处理器（CPU、GPU等）
-



编程接口



- 深度学习存在大量重复的基础算法、流程——重复工作。



深度学习框架发展历程



“石器时代”

~2012年：API不友好，没有GPU支持，尚不完善



MATLAB是由美国MathWorks公司出品的一种用于算法开发、数据分析以及数值计算的高级技术计算语言和交互式环境。



OpenNN 开发于 2003 年在国际工程数值方法中心的名为 RAMFLOOD的项目中是一个使用C++编写的开源类库。



Torch是Facebook的开源机器学习库、科学计算框架和基于Lua编程语言的脚本语言，是PyTorch的直系祖先。

深度学习框架发展历程

```
1 function [W1, W5, Wo] = MnistConv(W1, W5, Wo, X, D)
2 %
3 alpha = 0.01;
4 beta = 0.95;
5 momentum1 = zeros(size(W1));
6 momentum5 = zeros(size(W5));
7 momentum0 = zeros(size(Wo));
8 N = length(D);
9 bsize = 100;
10 blist = 1:bsize:(N-bsize+1);
11 % One epoch Loop
12 for batch = 1:length(blist)
13     dW1 = zeros(size(W1));
14     dW5 = zeros(size(W5));
15     dWo = zeros(size(Wo));
16     % Mini-batch loop
17     %
18     begin = blist(batch);
19     for k = begin:begin+bsize-
20         % Forward pass = inference
21         %
22         x = X(:, :, k);           % Input,           28x28
23         y1 = Conv(x, W1);          % Convolution,  20x20x20
24         y2 = ReLU(y1);
25         y3 = Pool(y2);             % Pool,         10x10x20
26         y4 = reshape(y3, [], 1);  %                2000
27         v5 = W5*y4;               % ReLU,         360
28         y5 = ReLU(v5);            %
29         v = Wo*y5;               % Softmax,      10
30     end
```

```
34     d(sub2ind(size(d), D(K), 1)) = 1;
35     % Backpropagation
36     %
37     e = d - y;                   % Output Layer
38     delta = e;
39     e5 = Wo' * delta;             % Hidden(ReLU) Layer
40     delta5 = (y5 > 0) .* e5;
41     e4 = W5' * delta5;           % Pooling Layer
42     e3 = reshape(e4, size(y3));
43     e2 = zeros(size(y2));
44     W3 = ones(size(y2)) / (2*2);
45     for c = 1:20
46         e2(:, :, c) = kron(e3(:, :, c), ones([2 2])) .* W3(:, :, c);
47     end
48     delta2 = (y2 > 0) .* e2;      % ReLU Layer
49     delta1_x = zeros(size(W1));   % Convolutional Layer
50     for c = 1:20
51         delta1_x(:, :, c) = conv2(x(:, :, c), rot90(delta2(:, :, c), 2), 'valid');
52     end
53     dW1 = dW1 + delta1_x;
54     dW5 = dW5 + delta5*y4';
55     dWo = dWo + delta *y5';
56 end
57 % Update weights
58 %
59 dW1 = dW1 / bsize;
60 dW5 = dW5 / bsize;
61 dWo = dWo / bsize;
62 momentum1 = alpha*dW1 + beta*momentum1;
63 W1 = W1 + momentum1;
64 momentum5 = alpha*dW5 + beta*momentum5;
65 W5 = W5 + momentum5;
66 momentum0 = alpha*dWo + beta*momentum0;
67 Wo = Wo + momentum0;
68 end
```

“青铜时代”

2013~2015年：CPU多核、GPU支持、不同编程风格

Caffe

Caffe是以C++/CUDA代码为主的深度学习框架，需要进行编译安装，支持命令行、Python和MATLAB接口，单机多卡、多机多卡等都可以很方便的使用

theano

Theano是一个Python库和优化编译器的开源项目，用于操作和评估数学表达式，尤其是矩阵值表达式。其计算是使用NumPy语法表示，并且编译后可在CPU/GPU架构上高效运行



Chainer是一个开源的深度学习框架，完全在 NumPy 和 CuPy Python 库的基础上用 Python 编写,因其采用边运行边定义 方案以及在大型系统上的性能而著称

深度学习框架发展历程

```
1  layer {
2      name: "mnist"
3      type: "Data"
4      transform_param {
5          scale: 0.00390625
6      }
7      data_param {
8          source: "mnist_train_lmdb"
9          backend: LMDB
10         batch_size: 64
11     }
12     top: "data"
13     top: "label"
14 }
15 layer {
16     name: "pool1"
17     type: "Pooling"
18     pooling_param {
19         kernel_size: 2
20         stride: 2
21         pool: MAX
22     }
23     bottom: "conv1"
24     top: "pool1"
25 }
```

```
# The train/test net protocol buffer definition
net: "examples/mnist/lenet_train_test.prototxt"
# test_iter specifies how many forward passes the test should carry out.
# In the case of MNIST, we have test batch size 100 and 100 test iterations,
# covering the full 10,000 testing images.
test_iter: 100
# Carry out testing every 500 training iterations.
test_interval: 500
# The base Learning rate, momentum and the weight decay of the network.
base_lr: 0.01
momentum: 0.9
weight_decay: 0.0005
# The Learning rate policy
lr_policy: "inv"
gamma: 0.0001
power: 0.75
# Display every 100 iterations
display: 100
# The maximum number of iterations
max_iter: 10000
# snapshot intermediate results
snapshot: 5000
snapshot_prefix: "examples/mnist/lenet"
```

“铁器时代”

2015~2018年：效率优化、分布式支持、蓬勃发展



PYTORCH



K Keras



CNTK

mxnet

飞桨
PaddlePaddle

“罗马时代”

2018~2019年：大规模训练、并行计算、可用性提升



“工业时代”

2020年~：多场景多任务支持、编译层优化、丰富的套件支持

- 基于编译器的算子优化
- 编程接口优化
- 自动分布式训练
- 全场景、异构设备支持
- 动态图、静态图融合
- 训练推理一体化





NEW ANNOUNCEMENTS

Catch up on the latest technical insights and tools from the PyTorch community.

[Read More](#) >

MEMBERSHIP AVAILABLE

Join the Membership that fits your goals.

[Join](#)

PYTORCH 2.2

PyTorch 2.2 offers ~2x performance improvement.

[Learn More](#)[PyTorch](#)[torchaudio](#)[torchtext](#)[torchvision](#)[torcharrow](#)[TorchData](#)[TorchRec](#)[TorchServe](#)[PyTorch on XLA Devices](#)

experiences
stability, and

深度学习框架的开发

[README](#) [Code of conduct](#) [Apache-2.0 license](#) [Security](#)



TensorFlow

python 3.9 | 3.10 | 3.11 | 3.12

python package 2.16.1

DOI 10.5281/zenodo.4724125

opencss best practices passing

opencss scorecard 8.2

oss-fuzz coverage failing

oss-fuzz build failing

OSSRank #9 (Top 1%)

Contributor Covenant v1.4 adopted

TF Official Continuous 8 passed, 0 failed

TF Official Nightly 13 passed, 3 failed

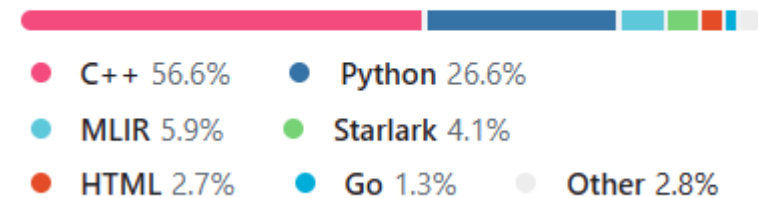
Documentation

api reference

[TensorFlow](#) is an end-to-end open source platform for machine learning. It has a comprehensive, flexible ecosystem of [tools](#), [libraries](#), and [community](#) resources that lets researchers push the state-of-the-art in ML and developers easily build and deploy ML-powered applications.

TensorFlow was originally developed by researchers and engineers working within the Machine Intelligence team at

Languages



 PyTorch

PyTorch is a Python package that provides two high-level features:

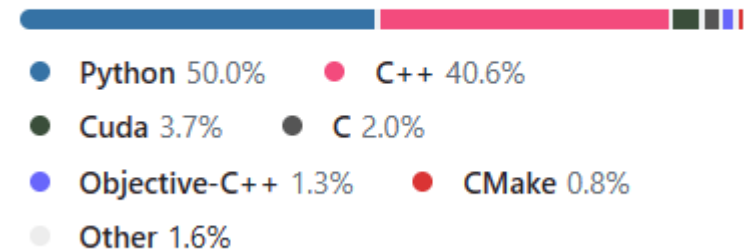
- Tensor computation (like NumPy) with strong GPU acceleration
- Deep neural networks built on a tape-based autograd system

You can reuse your favorite Python packages such as NumPy, SciPy, and Cython to extend PyTorch when needed.

Our trunk health (Continuous Integration signals) can be found at hud.pytorch.org.

- [More About PyTorch](#)
 - [A GPU-Ready Tensor Library](#)
 - [Dynamic Neural Networks: Tape-Based Autograd](#)
 - [Python First](#)
 - [Imperative Experiences](#)

Languages



深度学习框架的开发

- 现代C++
- CUDA
- C++代码封装（提供Python端接口）
- CMake——自动构建系统

深度学习框架的开发

- 由于Python的解释器是由C实现的，因此在Python中可以实现对于C和C++函数的调用。将底层C和C++函数生成对应的Python函数的过程过程被称为Python绑定。

手段主要包括：

- **Python的C-API**：要求在一个C++程序中包含Python.h，并用Python的C-API对Python进行操作。
- **Python的ctypes模块**：提供了C语言中的类型，以及直接调用动态链接库的能力。缺点是依赖于C的原生的类型，对自定义类型支持不好。
- **Boost::Python**：一个C++库，它可以将C++函数暴露为Python函数。其原理和Python C-API类似，但使用方法更简单。由于引入了Boost库，因此有沉重的第三方依赖。
- **Pybind11**：提供了类似于Boost::Python的简洁性和易用性，去除Boost依赖，因此成为了轻量级的Python库，从而特别适合在一个复杂的C++项目中暴露大量的Python函数。（现代机器学习框架，包括TensorFlow、PyTorch均依赖Pybind11）

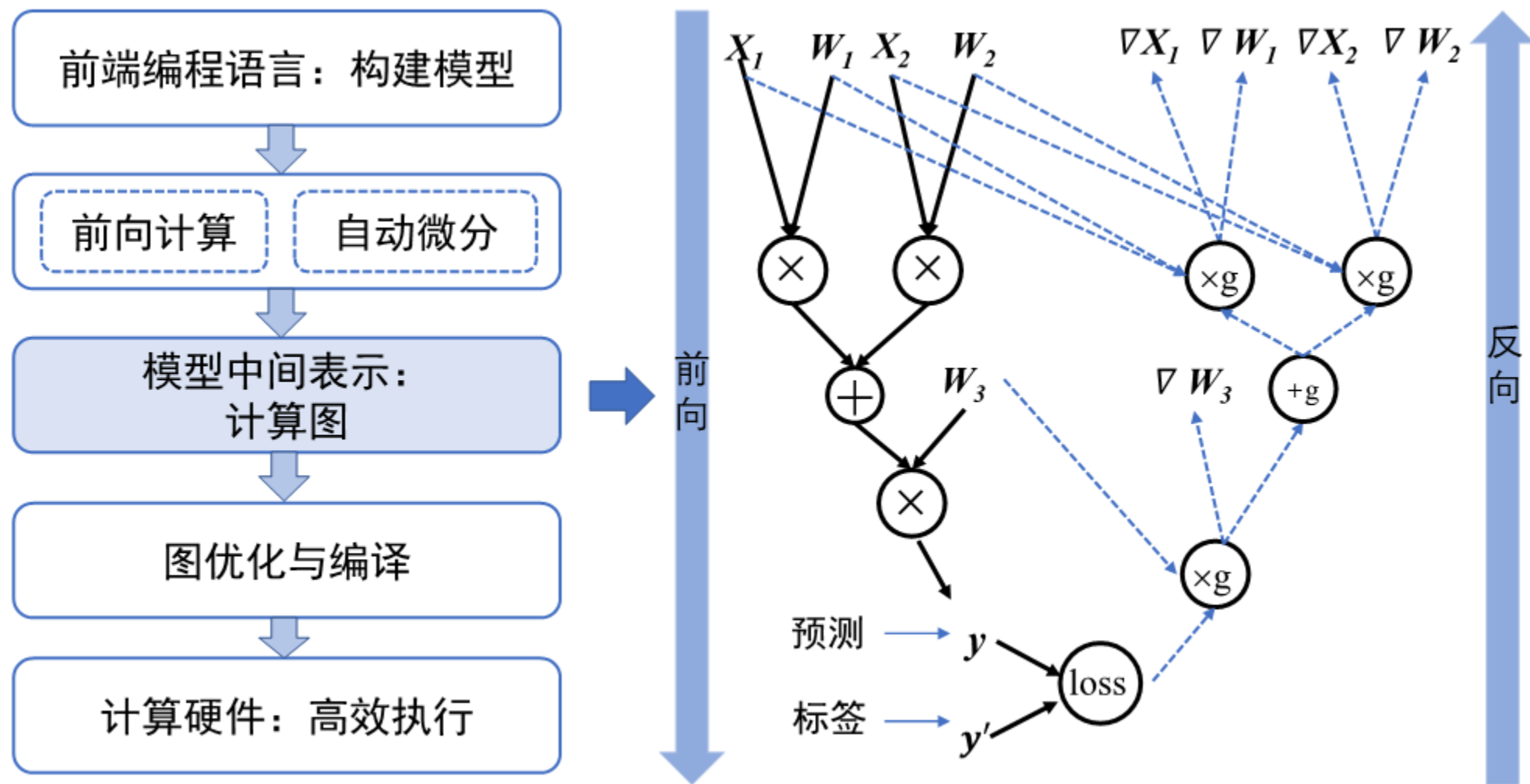
计算图



为什么需要计算图

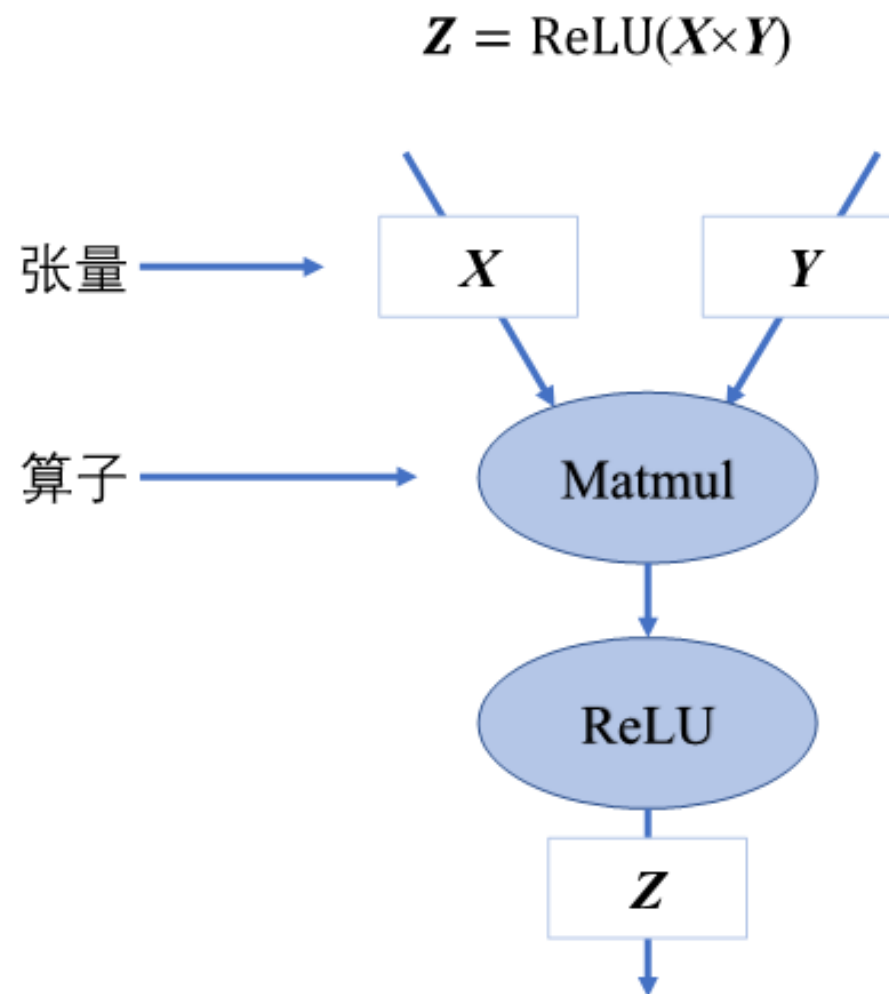
- 现代机器学习模型的拓扑结构日益复杂，机器学习系统需要一个通用的数据结构来理解、表达和执行机器学习模型。
- 计算图实现了以下关键功能：
 - 统一的计算过程表达。
 - 自动化计算梯度。
 - 分析模型变量生命周期，从而帮助框架优化内存管理。
 - 优化程序执行，分析模型结构和算子执行依赖关系，并自动寻找算子并行计算的策略，从而提高模型的执行效率。

计算图



计算图的组成

- 计算图由基本数据结构张量 (Tensor) 和基本运算单元算子构成。
- 在计算图中通常：
 - 使用节点来表示算子
 - 节点间的有向边来表示张量状态，同时也描述了计算间的依赖关系



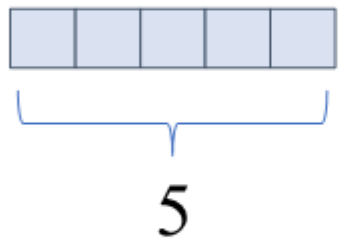
计算图的组成

● 张量：

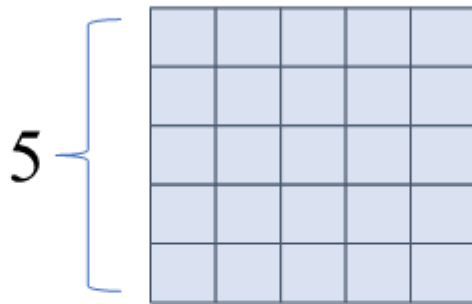
- 在机器学习领域，将多维数据称为张量
- 使用秩来表示张量的轴数或维度



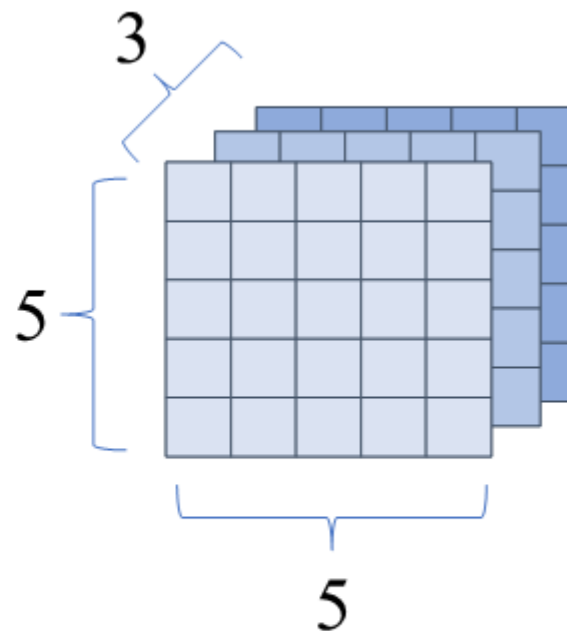
数据为5的标量



长度为5的
一维张量

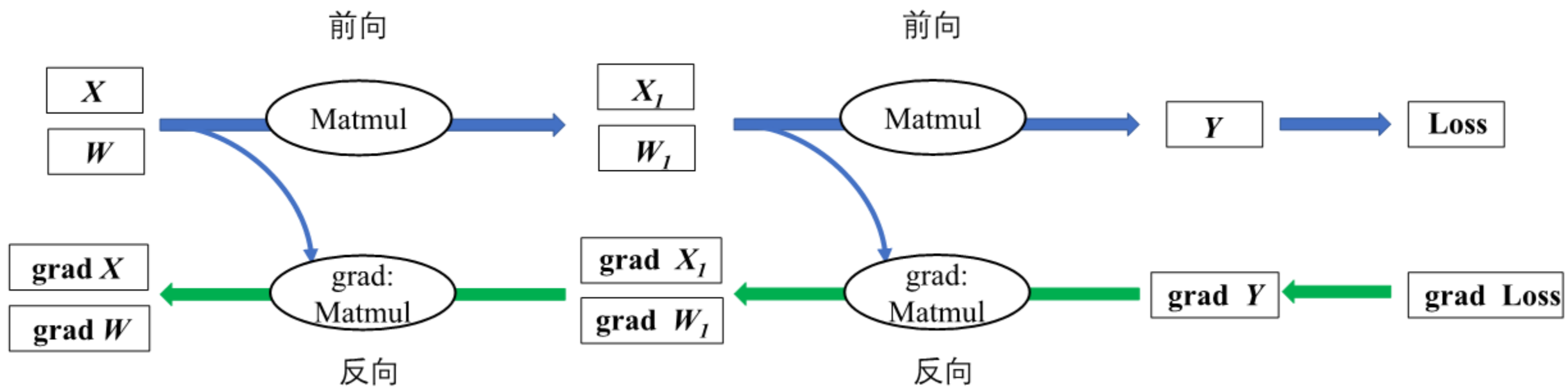


形状为 5×5 的
二维张量



形状为 $5 \times 5 \times 3$ 的
三维张量

计算图的组成



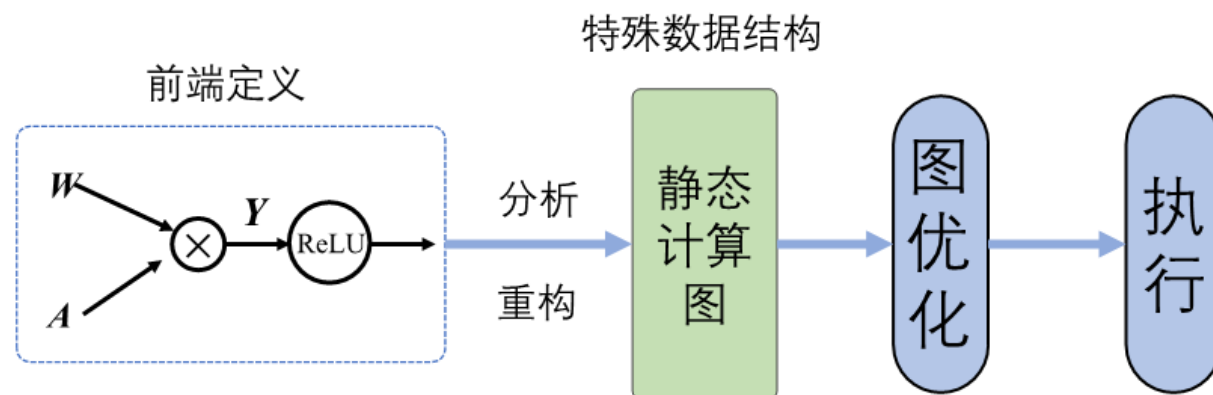
反向传播计算图

计算图的生成方式

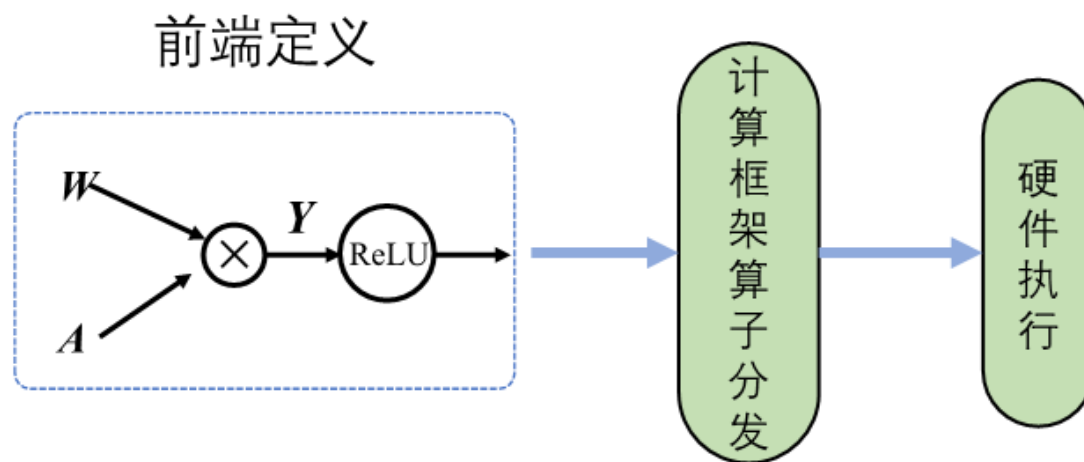
- 根据计算图的生成方式，可分为：

- 静态计算图
- 动态计算图

- 静态生成：先编译后执行



- 动态生成：采用解析式的执行方式，其核心特点是编译与执行同时发生。



计算图的生成方式

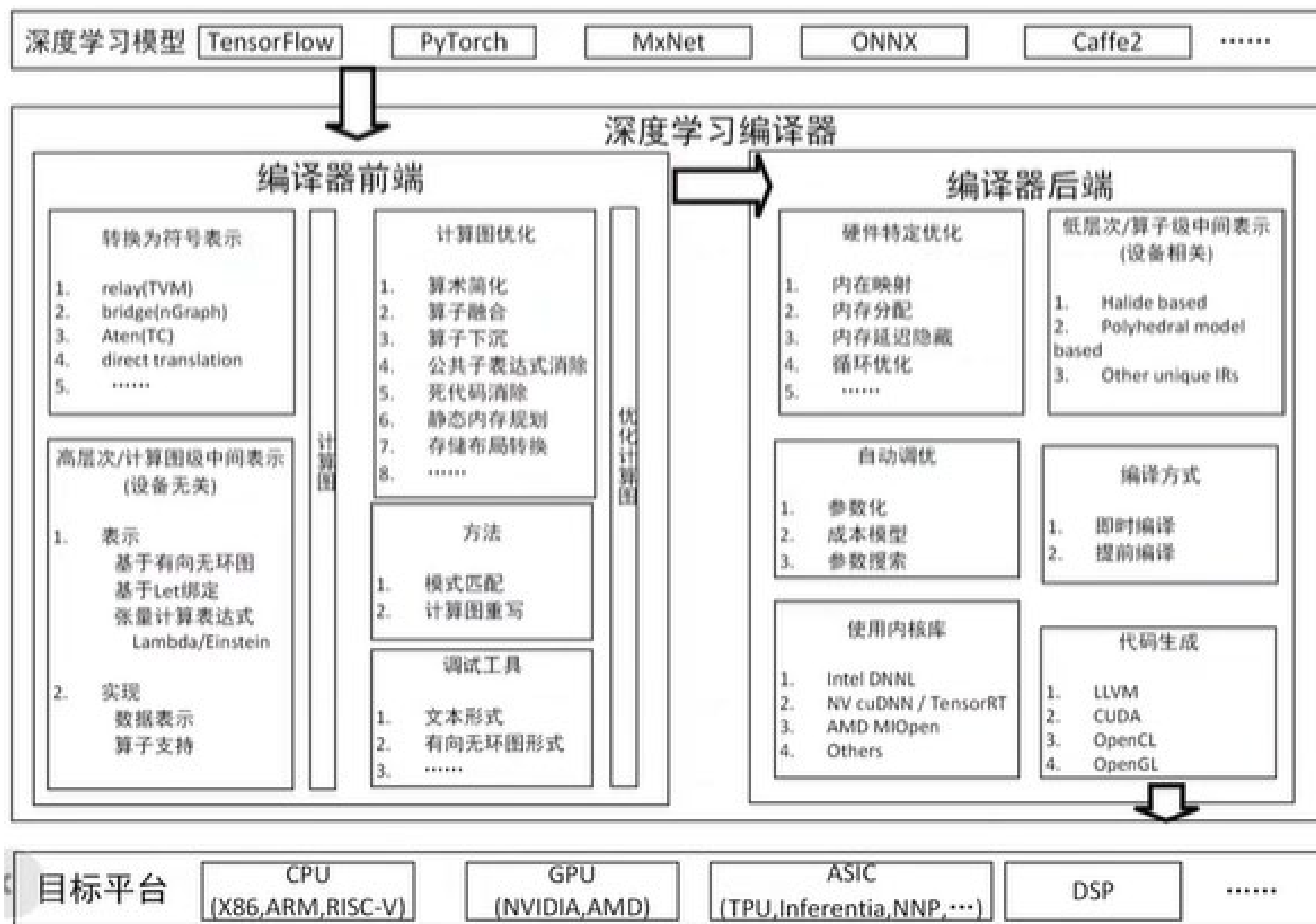
- 静态生成、动态生成对比

特性	静态图	动态图
即时获取中间结果	否	是
代码调试难易	难	易
控制流实现方式	特定的语法	前端语言语法
性能	优化策略多，性能更佳	图优化受限，性能较差
内存占用	内存占用少	内存占用相对较多
内存占用	可直接部署	不可直接部署

编译系统



编译系统



自动微分

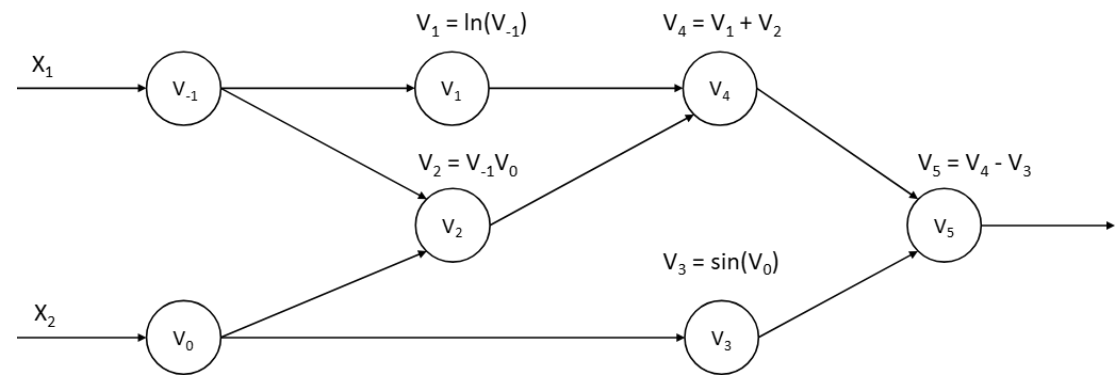
- 计算机程序中求导的方法：

- 手工微分：需手工求解函数导数的表达式
- 数值微分：通过差分近似方法完成，其本质是根据导数的定义推导而来 $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
- 符号微分：自动地通过数学规则对函数表达式进行递归变换
- 自动微分

- 自动微分 (Automatic Differentiation, AD) :

- 将计算机程序中的运算操作分解为一个有限的基本操作集合，且集合中基本操作的求导规则均为已知，在完成每一个基本操作的求导后，使用链式法则将结果组合得到整体程序的求导结果
- 在上个世纪六七十年代就已经被广泛应用于流体力学、天文学、数学金融等领域

自动微分



正向计算		
$V_{-1} = X_1$	$= 2$	
$V_0 = X_2$	$= 5$	
$V_1 = \ln V_{-1}$	$= \ln 2$	
$V_2 = V_{-1} \times V_0$	$= 10$	
$V_3 = \sin V_0$	$= \sin 5$	
$V_4 = V_1 + V_2$	$= 0.693 + 10$	
$V_5 = V_4 - V_3$	$= 10.693 + 0.959$	
$y = V_5 = V_4 - V_3$	$= 10.693 + 0.959$	

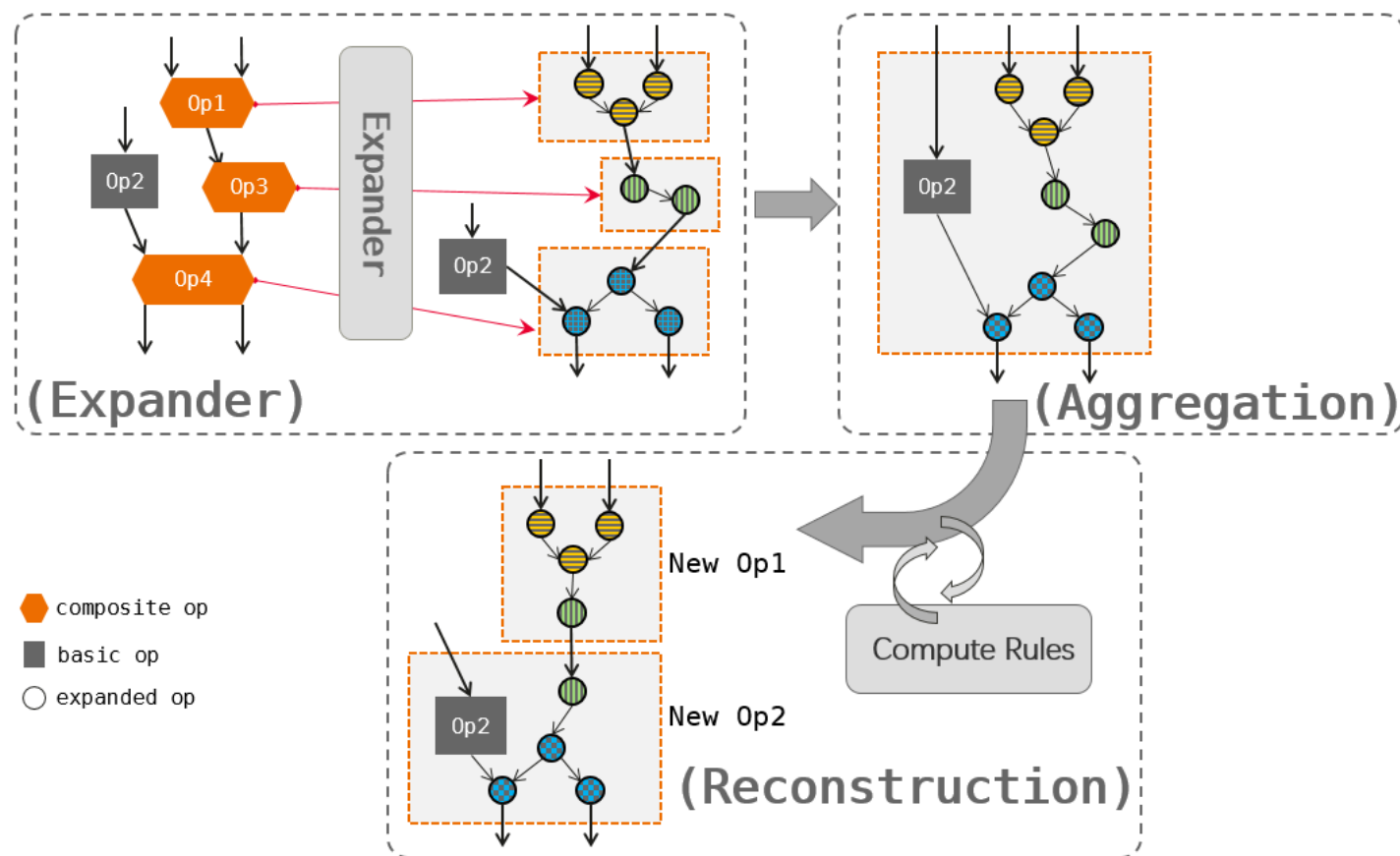
反向模式自动微分计算		
$\bar{X}_1 = \bar{V}_{-1}$	$= 5.5$	
$\bar{X}_2 = \bar{V}_0$	$= 1.716$	
$\bar{V}_{-1} = \bar{V}_{-1} + \bar{V}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{V}_{-1} + \frac{\bar{v}_1}{v_{-1}}$	$= 5.5$	
$\bar{V}_0 = \bar{V}_0 + \bar{V}_2 \frac{\partial v_2}{\partial v_0} = \bar{V}_0 + \bar{V}_2 \times V_{-1}$	$= 1.716$	
$\bar{V}_{-1} = \bar{V}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{V}_2 \times V_0$	$= 5$	
$\bar{V}_0 = \bar{V}_3 \frac{\partial v_3}{\partial v_0} = \bar{V}_2 \times \cos V_0$	$= -0.284$	
$\bar{V}_2 = \bar{V}_4 \frac{\partial v_4}{\partial v_2} = \bar{V}_4 \times 1$	$= 1$	
$\bar{V}_1 = \bar{V}_4 \frac{\partial v_4}{\partial v_1} = \bar{V}_4 \times 1$	$= 1$	
$\bar{V}_3 = \bar{V}_5 \frac{\partial v_5}{\partial v_3} = \bar{V}_5 \times (-1)$	$= -1$	
$\bar{V}_4 = \bar{V}_5 \frac{\partial v_5}{\partial v_4} = \bar{V}_5 \times 1$	$= 1$	
$\bar{V}_5 = \bar{y}$	$= 1$	

计算图优化

- 包括通用硬件优化、特定硬件优化

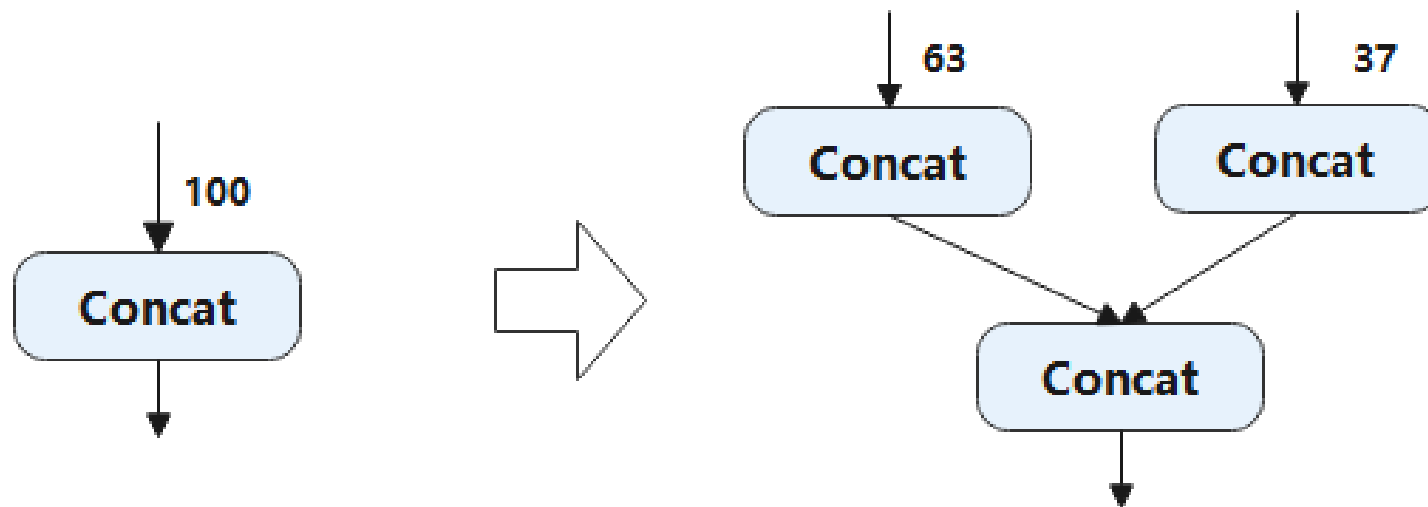
- 通用硬件优化：

- 在计算图中尝试匹配特定的子图结构，找到目标子图结构后，通过等价替换方式，将其替换成对硬件更友好的子图结构



计算图优化

- 包括通用硬件优化、特定硬件优化
- 特定硬件优化：
 - 包括由于硬件指令的限制而做的优化，特定硬件存储格式导致的优化等



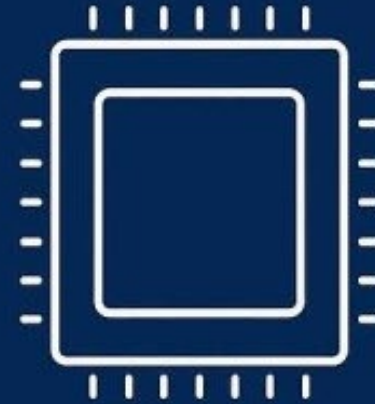
硬件加速器



CPU VS GPU



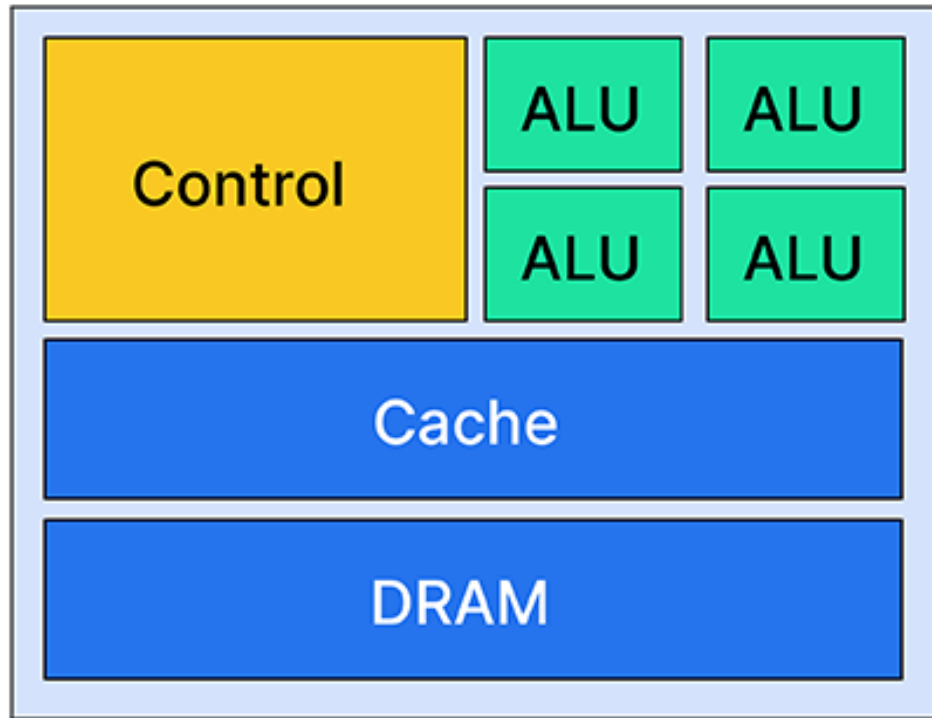
GPU



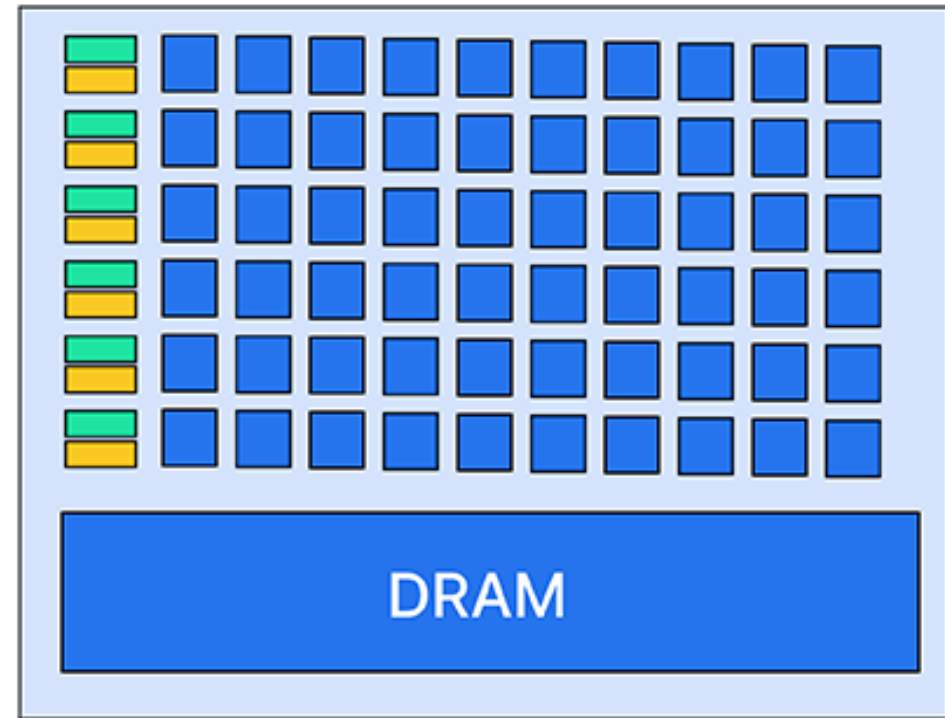
CPU

Purpose	Highly parallel tasks	General-purpose computing
Cores	Thousands of smaller, simpler cores	Few powerful cores (e.g. 4–32)
Performance	Optimized for parallel processing	Optimized for sequential processing
Use Cases	Matrix operations AI training, simulations	Running applications, managing processes
Latency vs Throughput	High throughput	Low latency

CPU VS GPU

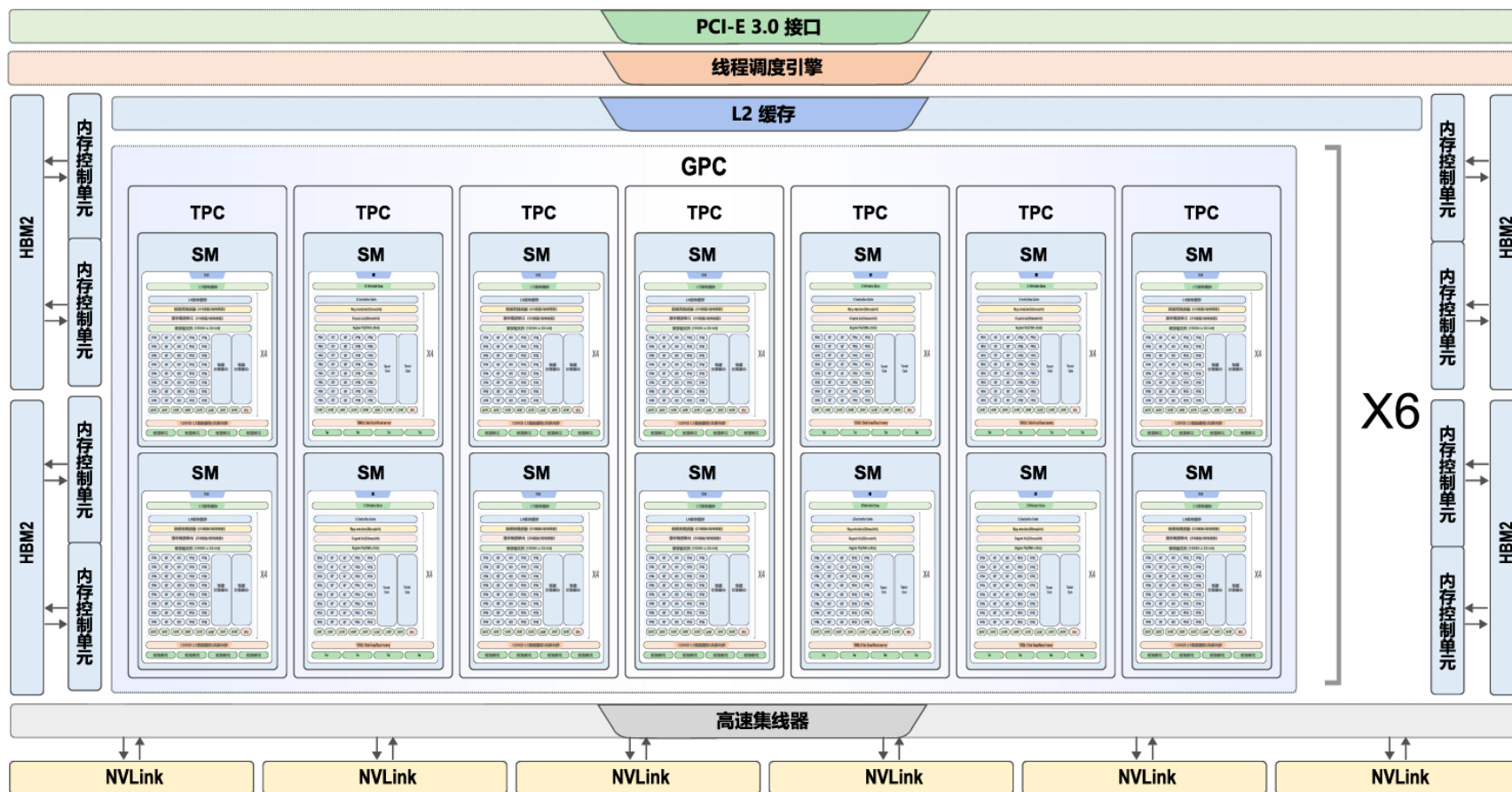


CPU



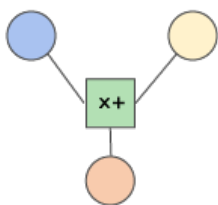
GPU

- 现代GPU在十分有限的面积上实现了极强的计算能力和极高的储存器以及IO带宽。
 - 以NVIDIA的Volta GV100为例，含有84个流处理器，5376个32位浮点运算单元，5376个32位整型运算单元，2688个64位浮点运算单元，672个张量运算单元和336个纹理单元。

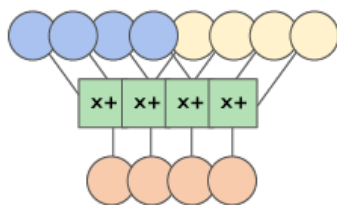


- 为了支持多种神经网络模型，现代GPU会设计多种计算单元

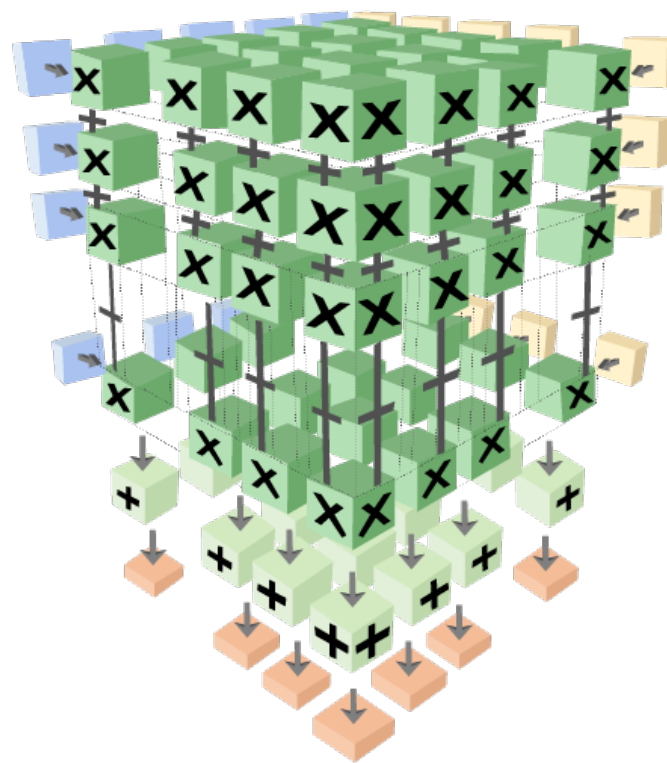
标量计算单元



一维向量计算单元

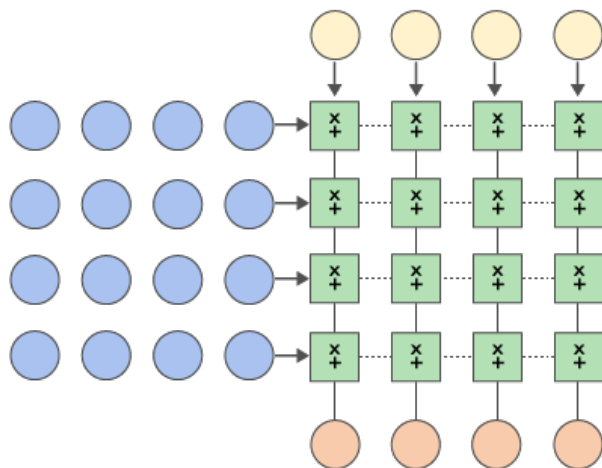


三维向量计算单元

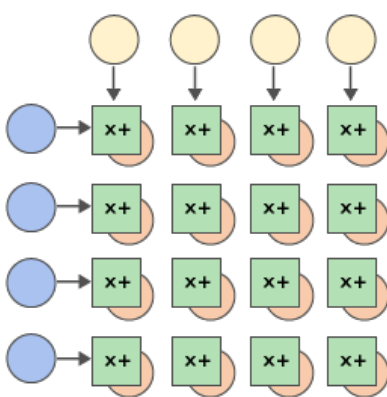


二维向量计算单元

点积



外积



● 算子库层级:

- 例如: NVIDIA提供了cuBLAS/cuDNN两类算子计算库, cuBLAS提供了使能张量计算核心的接口, 用以加速矩阵乘法(GEMM)运算, cuDNN提供了对应接口加速卷积(CONV)运算等

```
// 创建对象句柄, 设置数学计算模式
cublasHandle_t handle;
cublasStatus_t cublasStat = cublasCreate(&handle);
cublasStat = cublasSetMathMode(handle, CUBLAS_TENSOR_OP_MATH);

// 分配和初始化内存
size_t matrixSizeA = (size_t)M * K;
cublasStat = cudaMalloc(&devPtrA[0], matrixSizeA * sizeof(devPtrA[0][0]));
cublasStat = cublasSetMatrix(M, K, sizeof(A[0]), A, M, devPtrA[i], M);

// 调用算子
cublasStat = cublasGemmEx(handle, transa, transb, m, n, k, alpha,
                          A, CUDA_R_16F, lda,
                          B, CUDA_R_16F, ldb,
                          beta, C, CUDA_R_16F, ldc, CUDA_R_32F, algo);

// 传回结果数据, 释放内存、对象句柄
cublasStat = cublasGetMatrix(M, N, sizeof(D[0]), devPtrD[i], M, D, M);
cudaFree(devPtrA);
cudaDestroy(handle);
```

- 编程原语层级：

- 例如：通过在Device端调用CUDA WMMA (Warp Matrix Multiply Accumulate) API接口，以线程束（即{Warp}，是调度的基本单位）为操纵对象
- 在Volta架构下，针对FP16输入数据类型，NVIDIA官方提供了三种不同矩阵规模的WMMA乘累加计算接口，分别为 $16 \times 16 \times 16$ ， $32 \times 8 \times 16$ ， $8 \times 32 \times 16$
- 该API接口操纵的基本单位为Fragment，是一种指明了矩阵含义（乘法器/累加器）、矩阵形状（WMMA_M, WMMA_N, WMMA_K）、数据类型（FP16/FP32）、排布方式（row_major/col_major）等信息的模板类型

- 指令层级:

- 例如: 使用NVIDIA PTX指令集进行编程

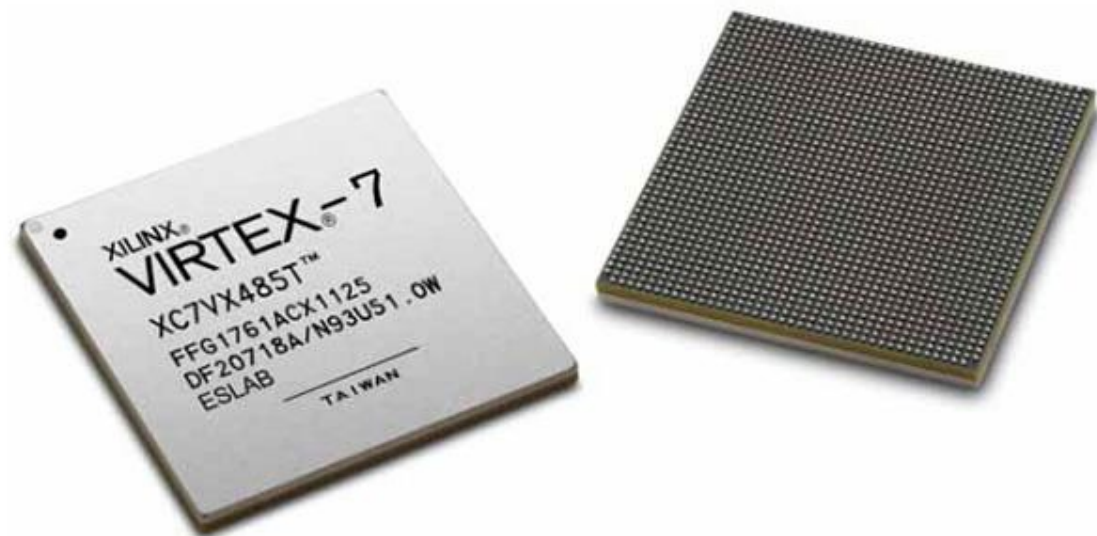
```
half_t *a, *b;
float *C, *D;
unsigned const* A = reinterpret_cast<unsigned const*>(a);
unsigned const* B = reinterpret_cast<unsigned const*>(b);

asm volatile(
    "mma.sync.aligned.m8n8k4.row.row.f32.f16.f16.f32 "
    "{%0,%1,%2,%3,%4,%5,%6,%7}, {%8,%9}, {%10,%11}, "
    "{%12,%13,%14,%15,%16,%17,%18,%19};\n"
    : "=f"(D[0]), "=f"(D[1]), "=f"(D[2]), "=f"(D[3]), "=f"(D[4]),
      "=f"(D[5]), "=f"(D[6]), "=f"(D[7])
    : "r"(A[0]), "r"(A[1]), "r"(B[0]), "r"(B[1]), "f"(C[0]),
      "f"(C[1]), "f"(C[2]), "f"(C[3]), "f"(C[4]), "f"(C[5]),
      "f"(C[6]), "f"(C[7]));
);
```


更多形式的硬件加速器

● FPGA：可编程阵列

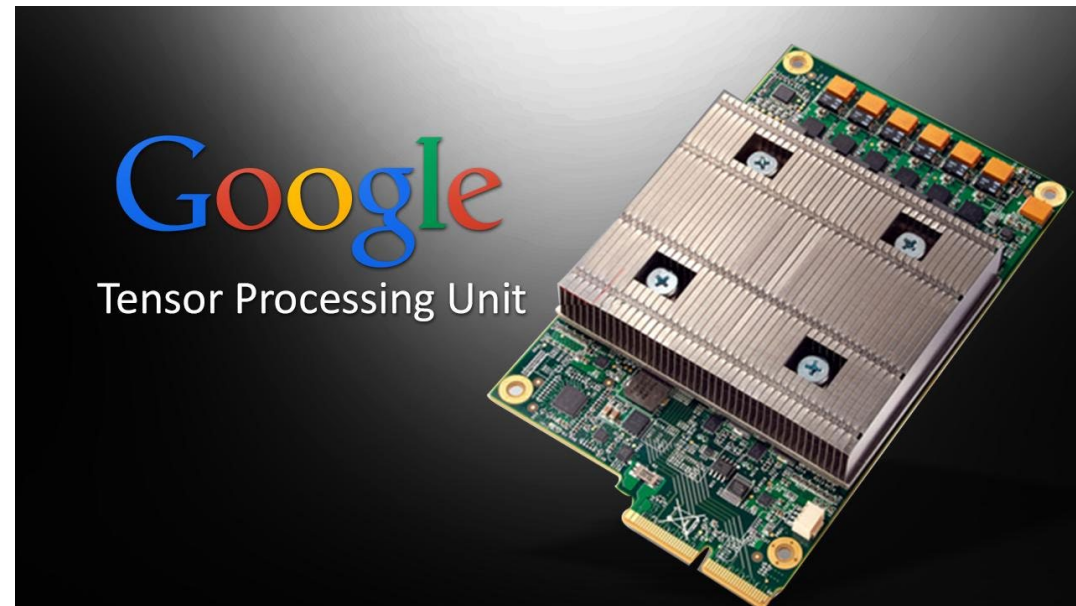
- 有大量可以编程的逻辑单元和可配置连接；
- 可以配置成计算复杂函数：
- 编程语言：VHDL、Verilog；
- 通常比CPU效率更高；
- 工具链质量良莠不齐；
- 一次“编译”需要数小时。



更多形式的硬件加速器

● ASIC：专用集成电路

- Google TPU是标志性芯片：
- 能够媲美NVIDIA GPU性能；
- 在Google大量部署；
- 核心是systolic array；
- 华为昇腾、寒武纪。



更多形式的硬件加速器



CPU



GPU



TPU



NPU



模型部署

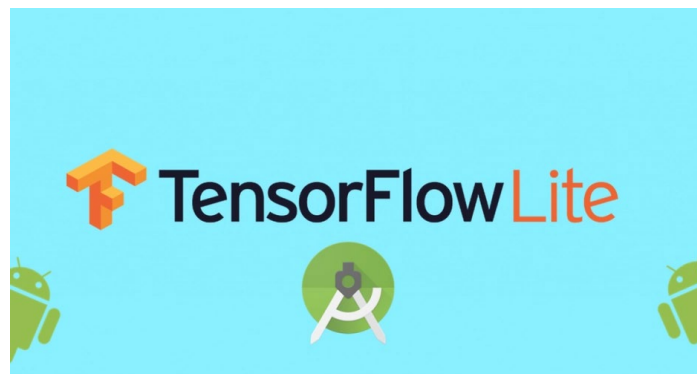


- **直接使用训练框架部署：如使用完整的TensorFlow、PyTorch等**

- 需要安装整个框架；
- 推理性能差；
- 很多冗余功能；
- 内存占用大。

- **训练框架部署引擎：TensorFlow Serving、TensorFlow Lite、TorchServe等；**

- 只支持自己框架训练的模型；
- 支持的硬件、OS有限。



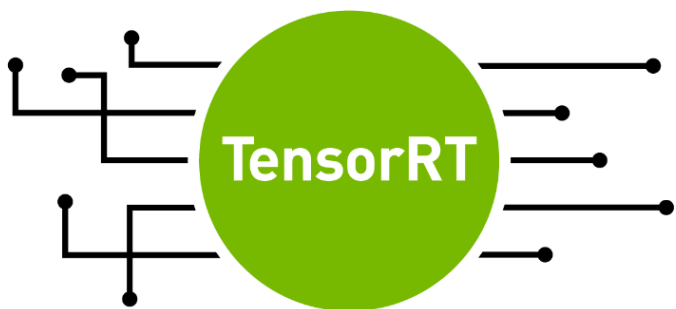
TORCHSERVE

TorchServe is a performant, flexible and easy to use tool for serving PyTorch

What's going on in TorchServe?

- High performance Llama 2 deployments with AWS Inferentia2 using TorchServe
- Naver Case Study: Transition From High-Cost GPUs to Intel CPUs and o
- Run multiple generative AI models on GPU using Amazon SageMaker m
75% in inference costs

- **手动模型重构：手写C/C++代码，实现计算图，导入权重；**
 - “从头造轮子”；
 - 需要开发者对模型有充分的了解；
 - 有较高的技术难度。
- **深度学习推理引擎：ONNX、OpenVINO、NVIDIA Tensor RT，移动端NCNN；**
 - 支持加载主流框架的模型文件；
 - 性能良好，能够满足不同环境下的应用需求。



- **不同场景的硬件对模型的要求不同**

- 比如部署在手机上，对于模型的大小比较敏感，一般在兆级别。
- 因此，对于一些较大的模型，往往需要通过一些模型压缩的技术，使其能满足不同计算硬件的要求。

- **常见的模型压缩策略：**

- 量化
- 模型稀疏 / 剪枝
- 知识蒸馏——教师-学生神经网络

模型并行计算

- **基本思路：**

- 在训练和推理时，将一个小批量的计算切分到多个GPU（或其他加速器）来达到加速目的

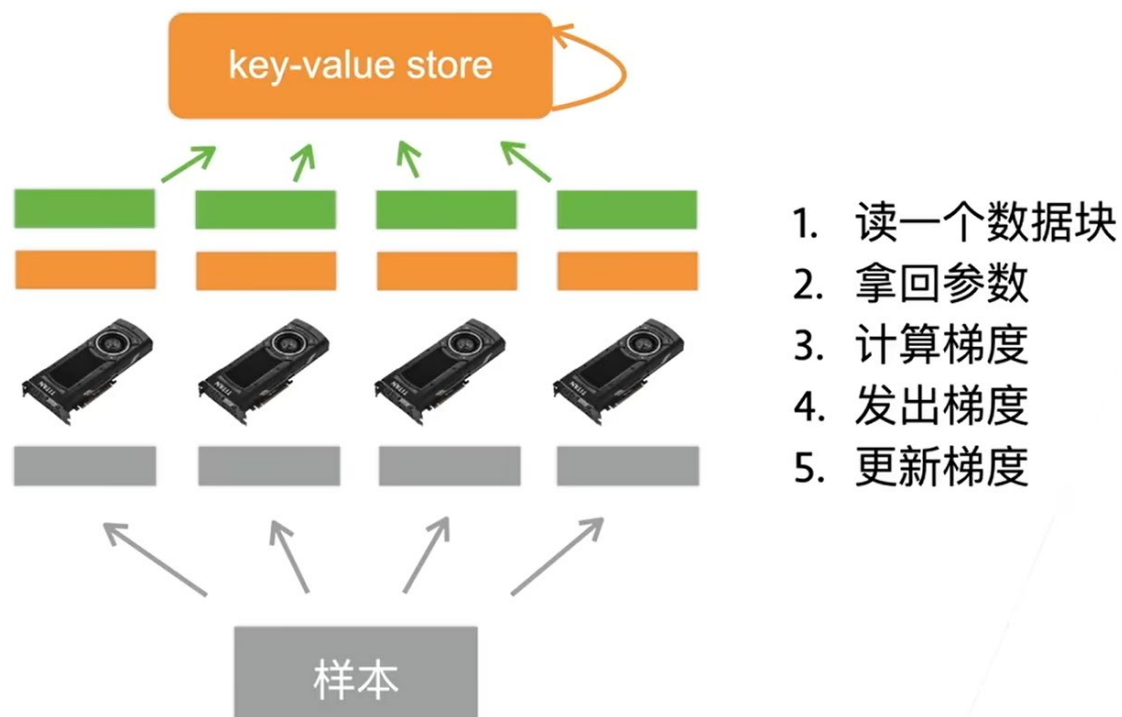
- **常用的切分方案：**

- 数据并行
- 模型并行
- 通道并行（数据+模型）

模型并行计算

● 数据并行:

- 将数据分成n块, 每个GPU拿到完整的参数计算数据的梯度



● 模型并行:

- 将模型分成n块, 每个GPU拿到一部分模型计算它的前向和反向结果

模型并行计算

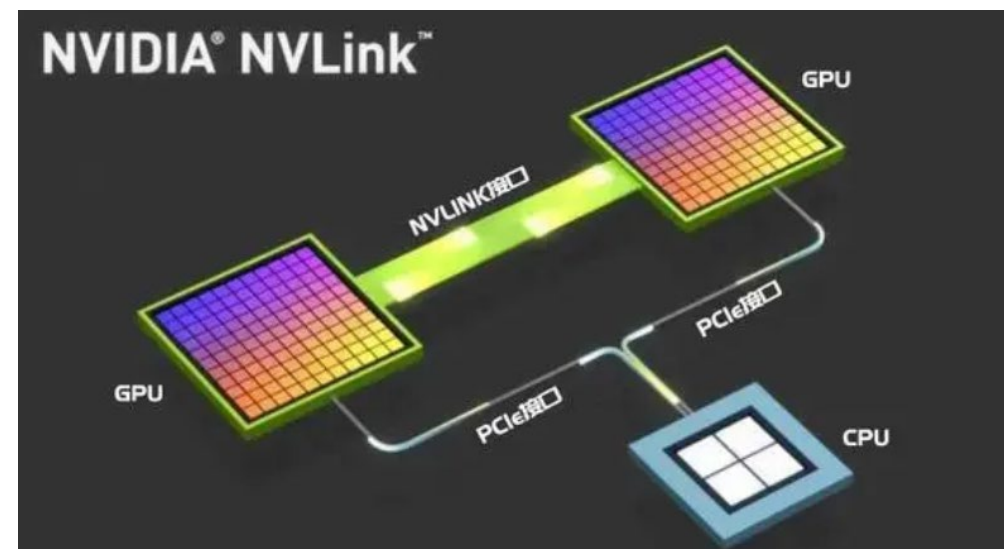
- 单机多卡场景的并行：

- PCIe：

- 一种基于串行通信的总线标准，采用点对点连接，支持多个设备同时进行高速数据传输；
- PCIe 4.0的带宽约为32GB/s；

- NVLink：

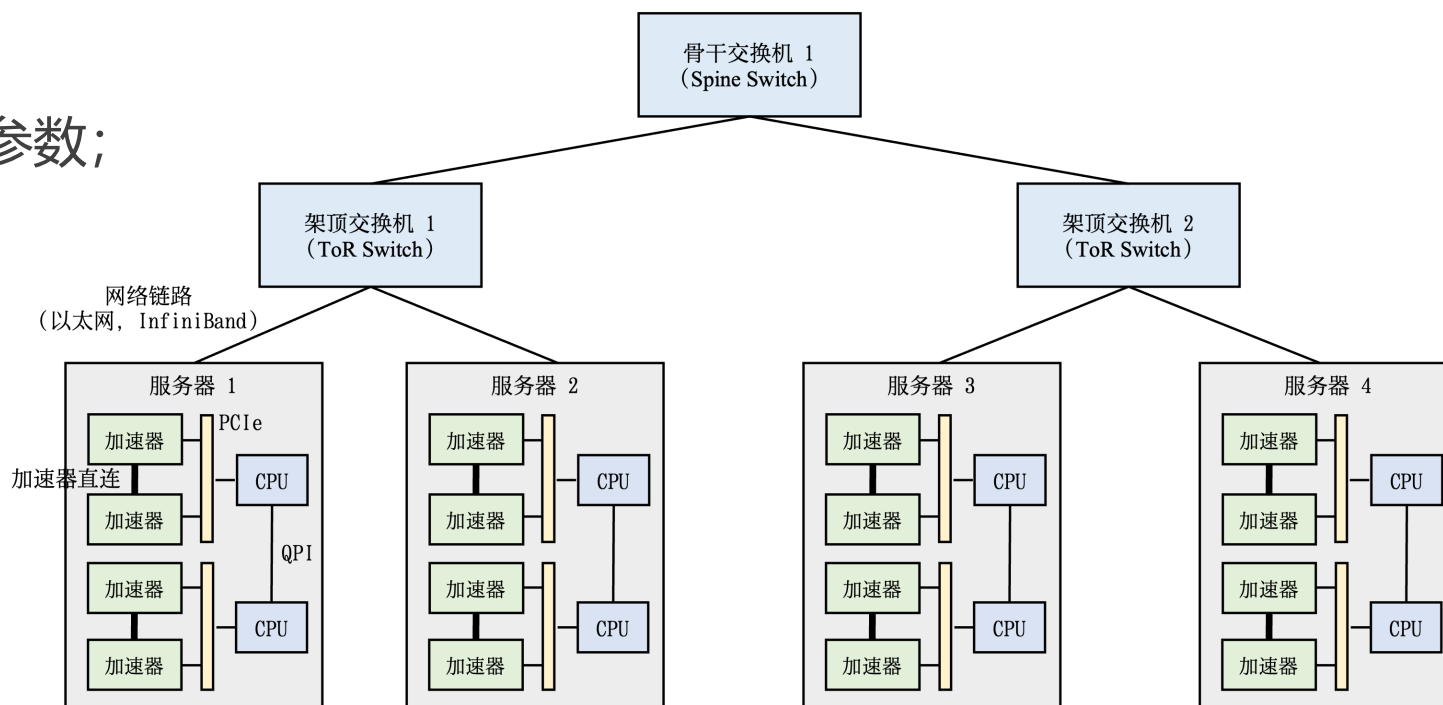
- NVidia开发的一种高速连接技术，主要用于连接GPU和其他组件，采用并行通信，较PCIe能提供更高的带宽和更低的延迟；
- NVLink 2.0的带宽高达300GB/s。



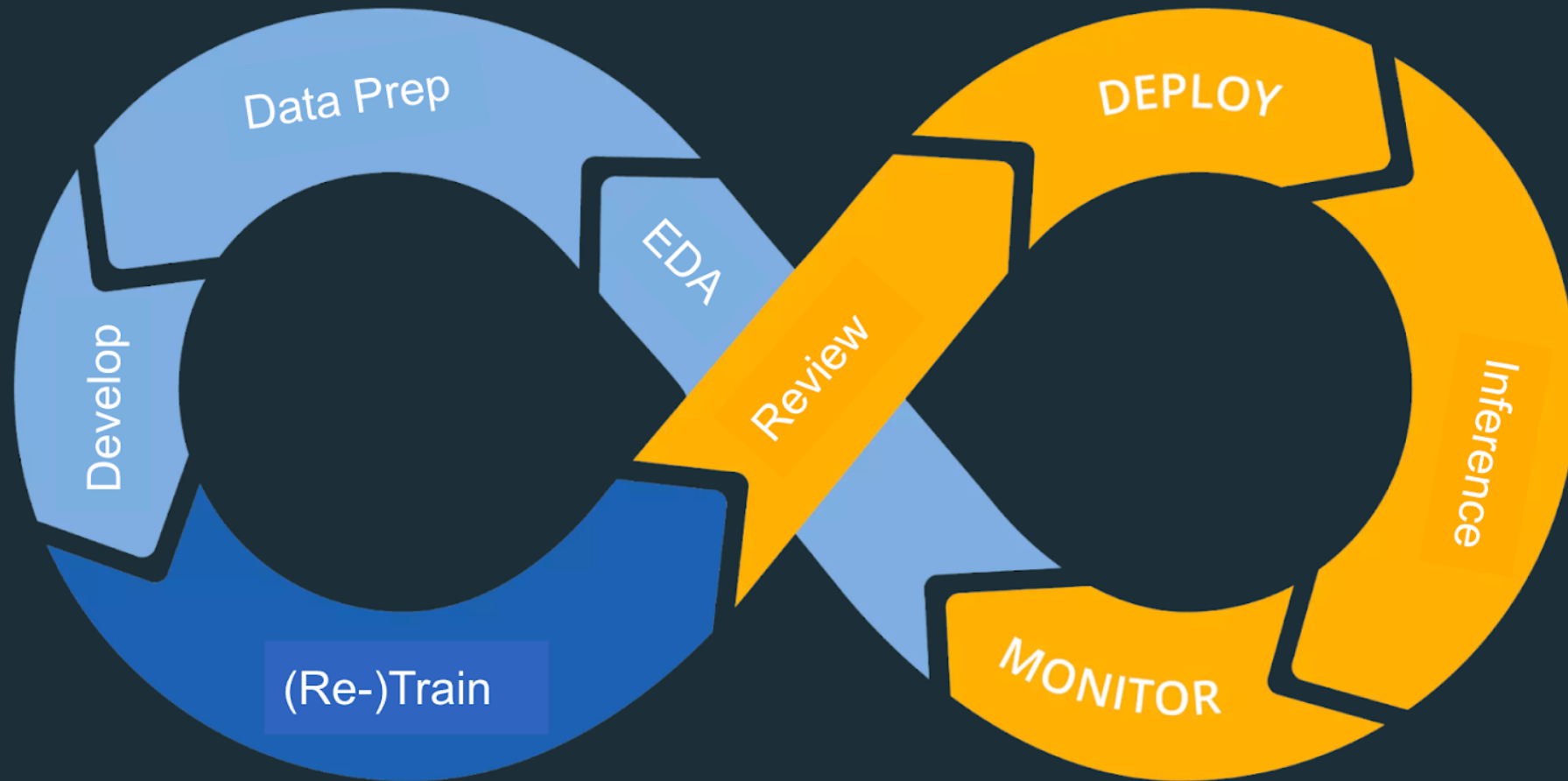
模型并行计算

● 多机多卡并行

- 每台计算服务器读取数据的一块;
- 进一步切分数据到每块GPU;
- 每个worker从参数服务器获取模型参数;
- 复制参数到每块GPU;
- 每块GPU计算梯度;
- 将所有GPU上的梯度求和;
- 梯度传回参数服务器;
- 每台服务器更新参数。



MLOps Cycle



Q&A